

```
-- Exercise 1
-- Rank employees by salary within each department (highest salary = rank 1).
-- Show name, department, salary, and their rank.
select concat(e.first_name, ' ', e.last_name) as employee_name, d.name as department_name, e.salary, rank() over (partition by d.id order by e.salary desc) as rank
from employees e join departments d
on e.department_id = d.id
```

employee_name varchar	department_name varchar	salary decimal(10,2)	rank int64
Iris Taylor	Marketing	105000.00	1
Uma Lopez	Marketing	78000.00	2
Tina Wright	Marketing	74000.00	3
Jack Anderson	Marketing	70000.00	4
Sam CEO	Engineering	200000.00	1
Alice Smith	Engineering	120000.00	2
Olivia Garcia	Engineering	92000.00	3
Bob Jones	Engineering	90000.00	4
Mia Harris	Engineering	88000.00	5
Carol White	Engineering	85000.00	6
.	.	.	.
.	.	.	.
.	.	.	.
David Brown	Sales	110000.00	1
Eva Davis	Sales	75000.00	2
Frank Miller	Sales	72000.00	3
Paula Lee	Sales	71000.00	4
Quinn Adams	Sales	69000.00	5
Nathan Martin	Sales	68000.00	6
Grace Wilson	HR	95000.00	1
Steve King	HR	67000.00	2
Henry Moore	HR	65000.00	3
Rachel Scott	HR	63000.00	4

26 rows (20 shown)

4 columns

```
-- Exercise 2
-- For each employee, show their salary and the difference from
-- the average salary of their department.
-- Label the difference column as 'diff_from_avg'.
select concat(e.first_name, ' ', e.last_name) as employee_name, d.name as department_name, e.salary, e.salary - avg(e.salary)
over (partition by d.id) as diff_from_avg
from employees e join departments d
on e.department_id = d.id
```

employee_name varchar	department_name varchar	salary decimal(10,2)	diff_from_avg double
Grace Wilson	HR	95000.00	22500.0
Henry Moore	HR	65000.00	-7500.0
Rachel Scott	HR	63000.00	-9500.0
Steve King	HR	67000.00	-5500.0
David Brown	Sales	110000.00	32500.0
Eva Davis	Sales	75000.00	-2500.0
Frank Miller	Sales	72000.00	-5500.0
Nathan Martin	Sales	68000.00	-9500.0
Paula Lee	Sales	71000.00	-6500.0
Quinn Adams	Sales	69000.00	-8500.0
.	.	.	.
.	.	.	.
.	.	.	.
Bob Jones	Engineering	90000.00	-14500.0
Carol White	Engineering	85000.00	-19500.0
Mia Harris	Engineering	88000.00	-16500.0
Olivia Garcia	Engineering	92000.00	-12500.0
Xander Baker	Engineering	82000.00	-22500.0
Yara Nelson	Engineering	79000.00	-25500.0
Karen Thomas	Finance	115000.00	29000.0
Leo Jackson	Finance	80000.00	-6000.0
Victor Hill	Finance	76000.00	-10000.0
Wendy Green	Finance	73000.00	-13000.0

26 rows (20 shown)

4 columns

```
-- Exercise 3
-- Using a CTE, find the top 2 highest-paid employees in each department.
with top_employees as (
  select concat(e.first_name, ' ', e.last_name) as employee_name, d.name as department_name, e.salary, rank()
  over (partition by d.id order by e.salary desc ) as rank
  from employees e join departments d
  on e.department_id = d.id
)
select * from top_employees
where rank <= 2
```

employee_name varchar	department_name varchar	salary decimal(10,2)	rank int64
David Brown	Sales	110000.00	1
Eva Davis	Sales	75000.00	2
Karen Thomas	Finance	115000.00	1
Leo Jackson	Finance	80000.00	2
Sam CEO	Engineering	200000.00	1
Alice Smith	Engineering	120000.00	2
Grace Wilson	HR	95000.00	1
Steve King	HR	67000.00	2
Iris Taylor	Marketing	105000.00	1
Uma Lopez	Marketing	78000.00	2

10 rows

4 columns

```
-- Exercise 4
-- Show a running total of sales amount ordered by sale_date.
-- Show: sale_date, amount, and running_total.
select sale_date, amount, sum(amount) over (order by sale_date) as running_total
from sales
order by sale_date
```

sale_date date	amount decimal(10,2)	running_total decimal(38,2)
2023-01-05	15000.00	15000.00
2023-01-12	18000.00	33000.00
2023-01-20	11000.00	44000.00
2023-01-25	13500.00	57500.00
2023-02-08	22000.00	79500.00
2023-02-14	9500.00	89000.00
2023-02-22	16000.00	105000.00
2023-03-10	27000.00	132000.00
2023-03-18	21000.00	153000.00
2023-03-25	12000.00	165000.00
.	.	.
.	.	.
.	.	.
2023-09-18	19000.00	427500.00
2023-09-27	13000.00	440500.00
2023-10-09	10000.00	450500.00
2023-10-23	26000.00	476500.00
2023-11-06	18000.00	494500.00
2023-11-19	22000.00	516500.00
2023-11-28	12000.00	528500.00
2023-12-05	32000.00	560500.00
2023-12-18	9000.00	569500.00
2023-12-29	14000.00	583500.00

34 rows (20 shown)

3 columns

```
-- Exercise 5
-- For each employee, show their hire_date and the hire_date of the
-- person hired just before them (using LAG).
select concat(e.first_name, ' ', e.last_name) as employee_name, e.hire_date, lag(e.hire_date) over (order by e.hire_date) as previous_hire_date
from employees e
order by e.hire_date
```

employee_name varchar	hire_date date	previous_hire_date date
Sam CEO	2010-01-01	NULL
Karen Thomas	2015-05-18	2010-01-01
Grace Wilson	2016-09-30	2015-05-18
David Brown	2017-11-20	2016-09-30
Alice Smith	2018-03-15	2017-11-20
Uma Lopez	2018-08-20	2018-03-15
Olivia Garcia	2018-12-01	2018-08-20
Steve King	2019-04-17	2018-12-01
Bob Jones	2019-06-01	2019-04-17
Iris Taylor	2019-08-22	2019-06-01
.	.	.
.	.	.
.	.	.
Frank Miller	2021-07-15	2021-04-05
Xander Baker	2021-09-05	2021-07-15
Rachel Scott	2021-11-30	2021-09-05
Henry Moore	2022-02-14	2021-11-30
Paula Lee	2022-03-08	2022-02-14
Wendy Green	2022-07-11	2022-03-08
Nathan Martin	2022-09-12	2022-07-11
Jack Anderson	2023-01-03	2022-09-12
Yara Nelson	2023-03-14	2023-01-03
Quinn Adams	2023-05-22	2023-03-14

26 rows (20 shown)

3 columns

```
-- Exercise 6
-- Categorize employees into salary bands using CASE:
-- 'Junior' : salary < 75,000
-- 'Mid' : salary between 75,000 and 99,999
-- 'Senior' : salary >= 100,000
-- Show name, salary, and band. Count how many fall in each band.
select concat(first_name, ' ', last_name) as employee_name, salary,
case
    when salary < 75000 then 'Junior'
    when salary between 75000 and 99999 then 'Mid'
    else 'Senior'
end as band,
count(id) over (partition by case
    when salary < 75000 then 'Junior'
    when salary between 75000 and 99999 then 'Mid'
    else 'Senior'
end) as employee_count
from employees
```

employee_name varchar	salary decimal(10,2)	band varchar	employee_count int64
Sam CEO	200000.00	Senior	5
Alice Smith	120000.00	Senior	5
David Brown	110000.00	Senior	5
Grace Wilson	95000.00	Mid	11
Iris Taylor	105000.00	Senior	5
Karen Thomas	115000.00	Senior	5
Bob Jones	90000.00	Mid	11
Eva Davis	75000.00	Mid	11
Leo Jackson	80000.00	Mid	11
Carol White	85000.00	Mid	11
.	.	.	.
.	.	.	.
.	.	.	.
Henry Moore	65000.00	Junior	10
Rachel Scott	63000.00	Junior	10
Steve King	67000.00	Junior	10
Jack Anderson	70000.00	Junior	10
Tina Wright	74000.00	Junior	10
Uma Lopez	78000.00	Mid	11
Victor Hill	76000.00	Mid	11
Wendy Green	73000.00	Junior	10
Xander Baker	82000.00	Mid	11
Yara Nelson	79000.00	Mid	11

```
-- Exercise 7
-- Find the month-over-month sales growth for 2023.
-- Show: month, total_sales, previous_month_sales, and growth percentage.
-- Hint: use LAG and date functions.
select date_trunc('month', sale_date) as month, sum(amount) as total_sales,
lag(sum(amount)) over (order by date_trunc('month', sale_date)) as previous_month_sales,
round(((sum(amount) - lag(sum(amount)) over (order by date_trunc('month', sale_date))) / 1
) * 100, 2) as growth_percentage
from sales
where sale_date between '2023-01-01' and '2023-12-31'
group by date_trunc('month', sale_date)
order by month
```

month timestamp	total_sales decimal(38,2)	previous_month_sales decimal(38,2)	growth_percentage double
2023-01-01 00:00:00	57500.00	NULL	NULL
2023-02-01 00:00:00	47500.00	57500.00	-17.39
2023-03-01 00:00:00	60000.00	47500.00	26.32
2023-04-01 00:00:00	41500.00	60000.00	-30.83
2023-05-01 00:00:00	56000.00	41500.00	34.94
2023-06-01 00:00:00	44000.00	56000.00	-21.43
2023-07-01 00:00:00	36000.00	44000.00	-18.18
2023-08-01 00:00:00	51000.00	36000.00	41.67
2023-09-01 00:00:00	47000.00	51000.00	-7.84
2023-10-01 00:00:00	36000.00	47000.00	-23.4
2023-11-01 00:00:00	52000.00	36000.00	44.44
2023-12-01 00:00:00	55000.00	52000.00	5.77

12 rows

4 columns

```
-- Exercise 8
-- Using a recursive CTE, show the full management chain for employee id = 3.
-- Output should show each level: employee name -> their manager -> their manager's manager, etc.
with recursive management_chain as (
select e.id, concat(e.first_name, ' ', e.last_name) as employee_name, e.manager_id, 1 as level
from employees e
where e.id = 3
union all
select m.id, concat(m.first_name, ' ', m.last_name) as employee_name, m.manager_id, mc.level + 1
from employees m
join management_chain mc on m.id = mc.manager_id
)
select * from management_chain
order by level
```

id int32	employee_name varchar	manager_id int32	level int32
3	Carol White	2	1
2	Bob Jones	1	2
1	Alice Smith	16	3
16	Sam CEO	NULL	4

```

=====
-- Exercise 9
-- For each project, calculate what percentage of the total company budget it represents.
-- Show project name, budget, and budget_percentage rounded to 2 decimal places.
select p.name as project_name, p.budget, round((p.budget / sum(p.budget) over ()) * 100, 2) as budget_percentage
from projects p

```

project_name varchar	budget decimal(12,2)	budget_percentage double
Website Redesign	50000.00	9.09
Mobile App v2	120000.00	21.82
Data Migration	80000.00	14.55
CRM Integration	95000.00	17.27
Security Audit	30000.00	5.45
Analytics Dashboard	70000.00	12.73
HR Portal	45000.00	8.18
Finance Reporting	60000.00	10.91

```

=====
-- Exercise 10
-- Find employees who have been with the company for more than 5 years
-- and have a salary below the company-wide median salary.
-- Show their name, hire_date, and salary.
select concat(e.first_name, ' ', e.last_name) as employee_name, e.hire_date, e.salary
from employees e
where e.hire_date < current_date - interval '5 years'
and e.salary < (select percentile_cont(0.5) within group (order by salary) from employees)

```

employee_name varchar	hire_date date	salary decimal(10,2)
Eva Davis	2021-04-05	75000.00
Frank Miller	2021-07-15	72000.00
Steve King	2019-04-17	67000.00
Tina Wright	2020-06-15	74000.00
Uma Lopez	2018-08-20	78000.00
Victor Hill	2021-02-28	76000.00

```

-- Exercise 11
-- Show each sales rep's sales performance compared to the best performer in their region.
-- Show: name, region, their total sales, the region's top sales, and the gap.
select concat(e.first_name, ' ', e.last_name) as employee_name, s.region, sum(s.amount) as total_sales,
max(sum(s.amount)) over (partition by s.region) as region_top_sales,
max(sum(s.amount)) over (partition by s.region) - sum(s.amount) as gap
from employees e
join sales s on e.id = s.employee_id
group by e.id, e.first_name, e.last_name, s.region
order by s.region, total_sales desc

```

employee_name varchar	region varchar	total_sales decimal(38,2)	region_top_sales decimal(38,2)	gap decimal(38,2)
Nathan Martin	East	146000.00	146000.00	0.00
Eva Davis	East	137500.00	146000.00	8500.00
Quinn Adams	East	75500.00	146000.00	70500.00
Frank Miller	West	128500.00	128500.00	0.00
Paula Lee	West	96000.00	128500.00	32500.00

```
-- Exercise 12 (Challenge)
-- Write a query that shows, for each department:
--   - Department name
--   - Total headcount
--   - Total salary budget
--   - Number of active projects their employees are on
--   - Average hours logged per employee on projects
with department_summary as (
  select department_id, count(*) as headcount, sum(salary) as total_salary_budget
  from employees
  group by department_id
)
select d.name as department_name, ds.headcount, ds.total_salary_budget,
count(distinct p.id) as active_projects,
round(avg(ep.hours_logged), 2) as avg_hours_per_employee
from departments d
left join department_summary ds on d.id = ds.department_id
left join employees e on d.id = e.department_id
left join employee_projects ep on e.id = ep.employee_id
left join projects p on ep.project_id = p.id and p.status = 'active'
group by d.id, d.name, ds.headcount, ds.total_salary_budget
```

department_name varchar	headcount int64	total_salary_budget decimal(38,2)	active_projects int64	avg_hours_per_employee double
Marketing	4	327000.00	1	190.0
Finance	4	344000.00	2	152.5
Engineering	8	836000.00	2	215.42
Sales	6	465000.00	1	88.75
HR	4	290000.00	0	72.5

```
-- Exercise 13
-- Using NTILE, divide all employees into 4 salary quartiles.
-- Show name, salary, and which quartile (1=lowest, 4=highest) they fall in.
select concat(e.first_name, ' ', e.last_name) as employee_name, e.salary,
ntile(4) over (order by e.salary) as salary_quartile
from employees e
order by e.salary
```

employee_name varchar	salary decimal(10,2)	salary_quartile int64
Rachel Scott	63000.00	1
Henry Moore	65000.00	1
Steve King	67000.00	1
Nathan Martin	68000.00	1
Quinn Adams	69000.00	1
Jack Anderson	70000.00	1
Paula Lee	71000.00	1
Frank Miller	72000.00	2
Wendy Green	73000.00	2
Tina Wright	74000.00	2
.	.	.
.	.	.
.	.	.
Carol White	85000.00	3
Mia Harris	88000.00	3
Bob Jones	90000.00	3
Olivia Garcia	92000.00	3
Grace Wilson	95000.00	4
Iris Taylor	105000.00	4
David Brown	110000.00	4
Karen Thomas	115000.00	4
Alice Smith	120000.00	4
Sam CEO	200000.00	4

26 rows (20 shown)

3 columns

```
=====
-- Exercise 14
-- For each employee, calculate how many days they have been with the company
-- as of today. Show name, hire_date, and days_employed.
  select concat(e.first_name, ' ', e.last_name) as employee_name, e.hire_date,
    current_date - e.hire_date as days_employed
  from employees e
  order by days_employed desc
```

employee_name varchar	hire_date date	days_employed int64
Sam CEO	2010-01-01	6058
Karen Thomas	2015-05-18	4095
Grace Wilson	2016-09-30	3594
David Brown	2017-11-20	3178
Alice Smith	2018-03-15	3063
Uma Lopez	2018-08-20	2905
Olivia Garcia	2018-12-01	2802
Steve King	2019-04-17	2665
Bob Jones	2019-06-01	2620
Iris Taylor	2019-08-22	2538
.	.	.
.	.	.
.	.	.
Frank Miller	2021-07-15	1845
Xander Baker	2021-09-05	1793
Rachel Scott	2021-11-30	1707
Henry Moore	2022-02-14	1631
Paula Lee	2022-03-08	1609
Wendy Green	2022-07-11	1484
Nathan Martin	2022-09-12	1421
Jack Anderson	2023-01-03	1308
Yara Nelson	2023-03-14	1238
Quinn Adams	2023-05-22	1169

26 rows (20 shown)

3 columns

```

=====
-- Exercise 15
-- Using a CTE, find all employees who earn more than the average salary
-- of ALL employees (not just their department).
-- Then show what percentage of the total salary bill they represent.
    with above_average_employees as (
        select e.id, concat(e.first_name, ' ', e.last_name) as employee_name, e.salary
        from employees e
        where e.salary > (select avg(salary) from employees)
    )
    select employee_name, salary, round((salary / (select sum(salary) from employees)) * 100, 2
    from above_average_employees

```

employee_name varchar	salary decimal(10,2)	percentage_of_total_salary double
Sam CEO	200000.00	8.84
Alice Smith	120000.00	5.31
David Brown	110000.00	4.86
Grace Wilson	95000.00	4.2
Iris Taylor	105000.00	4.64
Karen Thomas	115000.00	5.08
Bob Jones	90000.00	3.98
Mia Harris	88000.00	3.89
Olivia Garcia	92000.00	4.07

```

-- Exercise 16
-- Show the first sale and the last sale (by date) made by each sales employee.
-- Show: name, first_sale_date, first_sale_amount, last_sale_date, last_sale_amount.
-- Use window functions (FIRST_VALUE / LAST_VALUE or ROW_NUMBER).
    select distinct concat(e.first_name, ' ', e.last_name) as employee_name,
        first_value(s.sale_date) over (partition by e.id order by s.sale_date) as first_sale_date,
        first_value(s.amount) over (partition by e.id order by s.sale_date) as first_sale_amount,
        last_value(s.sale_date) over (partition by e.id order by s.sale_date rows between unbounded preceding and unbounded following) as last_sale_date,
        last_value(s.amount) over (partition by e.id order by s.sale_date rows between unbounded preceding and unbounded following) as last_sale_amount
    from employees e
    join sales s on e.id = s.employee_id

```

employee_name varchar	first_sale_date date	first_sale_amount decimal(10,2)	last_sale_date date	last_sale_amount decimal(10,2)
Paula Lee	2023-01-25	13500.00	2023-11-19	22000.00
Quinn Adams	2023-02-22	16000.00	2023-11-28	12000.00
Eva Davis	2023-01-05	15000.00	2023-12-18	9000.00
Frank Miller	2023-01-12	18000.00	2023-12-29	14000.00
Nathan Martin	2023-01-20	11000.00	2023-12-05	32000.00


```

=====
-- Exercise 17
-- Find pairs of employees who are in the same department AND
-- were hired within 6 months of each other.
-- Show both employee names, department, and their hire dates.
select concat(e1.first_name, ' ', e1.last_name) as employee1_name,
       concat(e2.first_name, ' ', e2.last_name) as employee2_name,
       d.name as department_name, e1.hire_date as employee1_hire_date,
       e2.hire_date as employee2_hire_date

from employees e1
join employees e2 on e1.department_id = e2.department_id and e1.id < e2.id
join departments d on e1.department_id = d.id
where abs(date_diff('day', e1.hire_date, e2.hire_date)) <= 180

```

employee1_name varchar	employee2_name varchar	department_name varchar	employee1_hire_date date	employee2_hire_date date
Henry Moore	Rachel Scott	HR	2022-02-14	2021-11-30
Leo Jackson	Victor Hill	Finance	2020-10-07	2021-02-28
Carol White	Mia Harris	Engineering	2020-01-10	2020-03-25
Eva Davis	Frank Miller	Sales	2021-04-05	2021-07-15

```

=====
-- Exercise 18
-- Calculate a 3-month rolling average of total sales per month for 2023.
-- Show: month, monthly_total, rolling_avg_3_months.
select date_trunc('month', sale_date) as month, sum(amount) as monthly_total,
       round(avg(sum(amount)) over (order by date_trunc('month', sale_date) rows between 2 preceding and current row), 2) as rolling_avg_3_months
from sales
where sale_date between '2023-01-01' and '2023-12-31'
group by date_trunc('month', sale_date)
order by month

```

month timestamp	monthly_total decimal(38,2)	rolling_avg_3_months double
2023-01-01 00:00:00	57500.00	57500.0
2023-02-01 00:00:00	47500.00	52500.0
2023-03-01 00:00:00	60000.00	55000.0
2023-04-01 00:00:00	41500.00	49666.67
2023-05-01 00:00:00	56000.00	52500.0
2023-06-01 00:00:00	44000.00	47166.67
2023-07-01 00:00:00	36000.00	45333.33
2023-08-01 00:00:00	51000.00	43666.67
2023-09-01 00:00:00	47000.00	44666.67
2023-10-01 00:00:00	36000.00	44666.67
2023-11-01 00:00:00	52000.00	45000.0
2023-12-01 00:00:00	55000.00	47666.67

12 rows

3 columns

```

=====
-- Exercise 19
-- Using a CTE, identify employees who have been on projects
-- that have gone over their original planned end_date (end_date < today but status = 'active').
-- Show employee name, project name, and planned end_date.
    with overdue_projects as (
        select ep.employee_id, p.name as project_name, p.end_date
        from employee_projects ep
        join projects p on ep.project_id = p.id
        where p.end_date < current_date and p.status = 'active'
    )
    select concat(e.first_name, ' ', e.last_name) as employee_name, op.project_name, op.end_date
    from overdue_projects op
    join employees e on op.employee_id = e.id

```

employee_name varchar	project_name varchar	end_date date
Alice Smith	Mobile App v2	2024-02-28
Iris Taylor	Analytics Dashboard	2024-03-31
Karen Thomas	Finance Reporting	2024-06-30
Bob Jones	Mobile App v2	2024-02-28
Leo Jackson	Finance Reporting	2024-06-30
Carol White	Mobile App v2	2024-02-28
Mia Harris	Mobile App v2	2024-02-28
Olivia Garcia	Mobile App v2	2024-02-28
Jack Anderson	Analytics Dashboard	2024-03-31
Tina Wright	Analytics Dashboard	2024-03-31
Uma Lopez	Analytics Dashboard	2024-03-31
Victor Hill	Finance Reporting	2024-06-30
Wendy Green	Finance Reporting	2024-06-30
Xander Baker	Mobile App v2	2024-02-28
Leo Jackson	Analytics Dashboard	2024-03-31
Victor Hill	Analytics Dashboard	2024-03-31
Wendy Green	Analytics Dashboard	2024-03-31

17 rows

3 columns

```

=====
-- Exercise 20
-- For each employee, show their salary percentile rank within the company
-- (i.e. what percentage of employees earn less than them).
-- Show name, salary, and percentile_rank rounded to 2 decimal places.
-- Hint: use PERCENT_RANK().
    select concat(e.first_name, ' ', e.last_name) as employee_name, e.salary,
    round(percent_rank() over (order by e.salary) * 100, 2) as percentile_rank
    from employees e
    order by e.salary desc

```

employee_name varchar	salary decimal(10,2)	percentile_rank double
Sam CEO	200000.00	100.0
Alice Smith	120000.00	96.0
Karen Thomas	115000.00	92.0
David Brown	110000.00	88.0
Iris Taylor	105000.00	84.0
Grace Wilson	95000.00	80.0
Olivia Garcia	92000.00	76.0
Bob Jones	90000.00	72.0
Mia Harris	88000.00	68.0
Carol White	85000.00	64.0
.	.	.
.	.	.
.	.	.
Tina Wright	74000.00	36.0
Wendy Green	73000.00	32.0
Frank Miller	72000.00	28.0
Paula Lee	71000.00	24.0
Jack Anderson	70000.00	20.0
Quinn Adams	69000.00	16.0
Nathan Martin	68000.00	12.0
Steve King	67000.00	8.0
Henry Moore	65000.00	4.0
Rachel Scott	63000.00	0.0

26 rows (20 shown)

3 columns

```

=====
-- Exercise 21 (Challenge)
-- Build a full employee summary report. For each employee show:
--   - Full name
--   - Department name
--   - Manager name (or 'No Manager')
--   - Salary band (Junior / Mid / Senior)
--   - Salary rank within their department
--   - Number of projects they are on
--   - Total hours logged across all projects
--   - Total sales amount (0 if not in sales)
-- Order by department name, then salary rank.
with project_summary as (
    select ep.employee_id, count(distinct ep.project_id) as project_count, sum(ep.hours_logged) as total_hours
    from employee_projects ep
    group by ep.employee_id
),
sales_summary as (
    select s.employee_id, sum(s.amount) as total_sales
    from sales s
    group by s.employee_id
)
select concat(e.first_name, ' ', e.last_name) as employee_name, d.name as department_name,
coalesce(concat(m.first_name, ' ', m.last_name), 'No Manager') as manager_name,

case
    when e.salary < 75000 then 'Junior'
    when e.salary between 75000 and 99999 then 'Mid'
    else 'Senior'
end as salary_band,

rank() over (partition by d.id order by e.salary desc) as salary_rank,
coalesce(ps.project_count, 0) as number_of_projects,
coalesce(ps.total_hours, 0) as total_hours_logged,
coalesce(ss.total_sales, 0) as total_sales_amount

from employees e
left join departments d on e.department_id = d.id
left join employees m on e.manager_id = m.id

```

```

left join employees m on e.manager_id = m.id
left join project_summary ps on e.id = ps.employee_id
left join sales_summary ss on e.id = ss.employee_id
group by e.id, e.first_name, e.last_name, e.salary, d.id, d.name, m.first_name, m.last_name, ps.project_count, ps.total_hours, ss.total_sales
order by d.name, salary_rank

```

employee_name varchar	department_name varchar	manager_name varchar	salary_band varchar	salary_rank int64	number_of_projects int64	total_hours_logged int128	total_sales_amount decimal(38,2)
Sam CEO	Engineering		Senior	1	0	0	0.00
Alice Smith	Engineering	Sam CEO	Senior	2	1	10	0.00
Olivia Garcia	Engineering	Bob Jones	Mid	3	3	615	0.00
Bob Jones	Engineering	Alice Smith	Mid	4	2	550	0.00
Mia Harris	Engineering	Bob Jones	Mid	5	1	300	0.00
Carol White	Engineering	Bob Jones	Mid	6	3	690	0.00
Xander Baker	Engineering	Bob Jones	Mid	7	1	260	0.00
Yara Nelson	Engineering	Bob Jones	Mid	8	1	160	0.00
Karen Thomas	Finance	Sam CEO	Senior	1	1	15	0.00
Leo Jackson	Finance	Karen Thomas	Mid	2	2	380	0.00
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.
Iris Taylor	Marketing	Sam CEO	Senior	1	1	180	0.00
Uma Lopez	Marketing	Iris Taylor	Mid	2	1	195	0.00
Tina Wright	Marketing	Iris Taylor	Junior	3	1	175	0.00
Jack Anderson	Marketing	Iris Taylor	Junior	4	1	210	0.00
David Brown	Sales	Sam CEO	Senior	1	1	20	0.00
Eva Davis	Sales	David Brown	Mid	2	1	130	137500.00
Frank Miller	Sales	Eva Davis	Junior	3	1	110	128500.00
Paula Lee	Sales	Eva Davis	Junior	4	1	95	96000.00
Quinn Adams	Sales	Eva Davis	Junior	5	0	0	75500.00
Nathan Martin	Sales	Eva Davis	Junior	6	0	0	146000.00

26 rows (20 shown)

8 columns